Locate, Then Reason: End-to-End Patch Presence Testing in Stripped Binaries

HUAIJIN WANG*, Shandong University, China
TONGDA WANG, Shandong University, China
ZHIBO LIU, Nanjing University, China
YUE ZHANG, Shandong University, China
SHUAI WANG, Hong Kong University of Science and Technology, Hong Kong SAR, China
PENGFEI HU, Shandong University, China
JIANZHONG LIU, Shandong University, China
FANGTIAN ZHONG, Shandong University, China
SHAOHUA WANG, Central University of Finance and Economics, China

arXiv:2610.25071v1 [cs.CR] 18 Oct 2026

Binary patch presence testing (PPT) determines whether a security fix is present in a stripped binary. Prior systems often assume that patch-relevant code is known or successfully matched. Function-level matching is brittle to compilation changes, inlining, finite model context, and missing usage context; moreover, similarity suppresses the patch-sized distinction needed for the verdict. We present `PatchAlign`, an end-to-end binary PPT workflow, which performs *large-scale localization that maximizes recall economically, then the final verdict reasons for patch-specific evidence*. `PatchAlign` uses instruction-aligned retrieval and caller-callee context to accurately locate patch-relevant binary code pieces, then screens the shortlist and compares target binary code with vulnerable and patched references using an LLM. Our experiments indicate that `PatchAlign` can precisely localize patch-relevant code and determine patch status without a target-location oracle.

ACM Reference Format:
Huaijin Wang, Tongda Wang, Zhibo Liu, Yue Zhang, Shuai Wang, Pengfei Hu, Jianzhong Liu, Fangtian Zhong, and Shaohua Wang. 2026. Locate, Then Reason: End-to-End Patch Presence Testing in Stripped Binaries. 1, 1 (October 2026), 21 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Security fixes do not reach every downstream product at the same time [15, 43]. Vendors fork upstream projects and backport selected changes [43]. Their products may be available only as stripped binaries, without corresponding source releases or reliable patching information [15, 24]. Binary patch presence testing (PPT) asks whether a particular security patch is present in such a

*Corresponding author

Authors' addresses: Huaijin Wang, Shandong University, China, huaijinwang@sdu.edu.cn; Tongda Wang, Shandong University, China, wangtongda@mail.sdu.edu.cn; Zhibo Liu, Nanjing University, China, zhiboliu@nju.edu.cn; Yue Zhang, Shandong University, China, zyueinfosec@gmail.com; Shuai Wang, Hong Kong University of Science and Technology, Hong Kong SAR, China, shuaiw@cse.ust.hk; Pengfei Hu, Shandong University, China, phu@sdu.edu.cn; Jianzhong Liu, Shandong University, China, liujianzhong@sdu.edu.cn; Fangtian Zhong, Shandong University, China, fzhong@sdu.edu.cn; Shaohua Wang, Central University of Finance and Economics, China, davidshwang@ieee.org.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM XXXX-XXXX/2026/10-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

binary and is therefore a basic capability for vulnerability assessment and patch management [15, 29, 38–40, 42].

Binary PPT is difficult because its signal is both small and unstable. A patch may change only a few source lines and compile into a handful of instructions, while the rest of the vulnerable and patched programs remains nearly identical [18, 38, 42]. At the same time, compiler choice and optimization can rewrite those instructions [16], reshape control flow [8, 34], and move a callee's code into its callers [2, 13]. An operational system must consequently solve two coupled problems: it must locate the code that implements the patch-related behavior and then decide whether that code implements the vulnerable or patched semantics [20, 38, 42].

End-to-End Binary PPT. Given a public source patch, source-mapped vulnerable and patched reference builds, and a stripped target binary, locate the patch-relevant code in the target—without an oracle for its function or address—and decide whether the target exhibits the vulnerable or patched behavior.

Several binary PPT systems perform patch verification on an affected target function that is supplied or identified beforehand [40–42]. Some take an affected function or entry address directly [40, 42]; others use a matching front end but cannot test a candidate that was never recovered [38, 39]. This condition is particularly restrictive under inlining: the affected source function can disappear as a standalone binary function even though its behavior remains inside a caller [15].

Binary code similarity analysis (BCSA) is a natural localization front end since it has been frequently applied to vulnerable function detection [5, 8, 20, 21, 33, 34, 44]. However, conventional function-level similarity does not directly solve end-to-end patch presence testing. Consider the tcpdump fix for CVE-2018-14461, which increases two bounds checks from 8 to 12 bytes [17]. In the optimized target binary, the affected function is inlined into a larger caller, so localization must recover its behavior within that caller [13]. Even after finding the correct code, coarse function-level similarity cannot establish whether both corrected checks protect the relevant accesses [27, 36]. This example motivates locating the relevant behavior first and then checking the patch-specific conditions.

Our Approach. We present `PatchAlign`, an end-to-end workflow that separates recall-oriented localization from patch-conditioned reasoning. Reference Patch Analysis derives patch-relevant functions and caller-callee contexts from known vulnerable and patched builds, and generates a reusable plan for distinguishing each pair's vulnerable and patched behavior. Semantic Localization searches CFG windows covering recovered functions and refines candidate rankings using instruction similarity and caller-callee context. Relevance Screening rejects incidental matches. Patch-Status Reasoning reuses the corresponding plan with both reference functions and the accepted target code, in assembly or decompiled code. The final binary verdict aggregates votes from all analyzed functions. The analysis pipeline requires neither symbols, source mappings, nor a location oracle of the target binary being analyzed.

We evaluate `PatchAlign` on a rebuilt, balanced version of the PS^3 benchmark [40]. The evaluation compares prior systems, separates abstention coverage from decided-case accuracy, and examines localization, challenging patches, robustness, component contributions, and cost.

This paper makes the following contributions:

- We identify limitations overlooked when binary PPT is treated only as function comparison: prior systems assume patch-relevant code is available, while function-level matching is brittle under compilation changes such as inlining; high similarity alone does not establish whether the security fix is present.

- We construct PatchAlign, an end-to-end binary PPT workflow that needs no target function or address. It combines recall-oriented, context-aware, and lightweighted semantic localization to locate patch-relevant binary code in the target binary; then it invokes relatively expensive reasoning model to patch presence reasoning.
- We perform a comprehensive evaluation for PatchAlign on a balanced benchmark with real-world CVEs against prior systems, covering end-to-end PPT, localization accuracy, and overhead measurement, demonstrating the essence of end-to-end PPT for binaries and the effectiveness of our new workflow.

2 RELATED WORK

2.1 Existing Binary Patch Presence Testing

Binary PPT determines whether a specific security patch is present in a target binary. Its inputs normally include the source-level patch, vulnerable and patched reference builds, and a target binary whose patch status is unknown; its output is a vulnerable or patched verdict. This task is narrower than general vulnerability search: it must recognize the small behavioral distinction introduced by one known patch.

Existing systems capture this distinction at several representations. Fiber derives compact syntactic and semantic signatures from source-level changes and searches for them in a localized target control flow graph (CFG) [42]. BinXray maps vulnerable and patched basic blocks and represents their difference with basic-block traces [38]. PDiff compares patch-related semantic summaries for downstream kernels [15], whereas Osprey uses inter-basic-block data-flow slices without symbolic execution [29]. Robin verifies candidates using constrained symbolic execution and concrete behavior [39]. More recent systems seek greater robustness across compilation configurations: PS^3 uses symbolic emulation and semantic signature equivalence [40], and React lifts binaries to an intermediate representation and checks refined semantic features with an SMT solver [41].

Despite this progress, three assumptions limit end-to-end use on stripped and optimized binaries.

C1: Assumed Patch-Code Location. Several systems either accept an affected target function or entry address or make patch classification conditional on an upstream match. For example, Fiber's evaluation uses kernel symbols and leaves similarity-search integration to future work, while PS^3 explicitly assumes that the analyzed function's entry address is known [40, 42]. BinXray and Robin include lightweight function-matching front ends, but its patch test still operates on the function selected by that stage [38]. Thus, not every system omits retrieval, but their patch-classification result generally presupposes that a suitable patch-relevant target function has been found.

C2: Compilation-Sensitive Evidence. Compiler choice and optimization can change instructions, data flow, and CFG structure even when source semantics are unchanged. Although PS^3, Robin, and React are designed to improve cross-configuration robustness, their evaluations report residual errors under compiler and optimization changes. Compilation sensitivity therefore remains a practical limitation rather than a solved problem [39–41].

C3: Affected Functions Lost to Optimization. Function-centric patch tests require a standalone target function on which to extract or match evidence. Optimization may instead inline the affected function into one or more callers and remove its entry entirely. In that case, knowing the source function name is insufficient because the expected binary object no longer exists. This case is excluded from the PS^3 dataset when the affected symbol disappears through inlining, merging, and specialization, and asymmetric inlining is known to invalidate one-to-one binary-function correspondence assumptions [13, 14, 40].

2.2 Binary Code Similarity Analysis

Binary code similarity analysis (BCSA) retrieves code that likely originates from the same or semantically similar source. Machine-learning-based (ML-based) methods encode binary functions as embedding vectors to support efficient queries. Representative designs include graph neural networks [11, 37], sequence and random-walk models [8, 21], and Transformers with control-flow-aware encoding [31, 35]. These methods are valuable as retrieval mechanisms, but their dominant function-level formulation has four limitations for PPT.

C4: One-to-One Matching under Inlining. Conventional BCSA treats recovered function boundaries as semantic boundaries and generally seeks a one-to-one match. With asymmetric inlining, the reference may be a standalone callee while the target representation combines that callee with a much larger caller. The compared embeddings then summarize different semantic scopes, and there may be no standalone target counterpart to retrieve [13]. `CI-Detector` [14] uses pairwise similarity models [20] to classify whether one binary function is cross-inlined into another. Unlike embedding-based approaches [31, 35, 36], it neither supports efficient distance-based retrieval nor captures broader semantic-containment relations, making it unsuitable for large-scale semantics containment detection [20, 32, 33].

C5: Finite Representation Context. Sequence encoders have a finite input window. Long functions—especially callers enlarged by inlining—may therefore be truncated before their patch-relevant instructions are encoded. `InsnAlign` reports retrieval failures when truncation at a 1,024-token limit discards the corresponding instructions in optimized functions [36]. This situation is common, Fig. 1 shows our study in PS^3 dataset [40]: For binaries optimized for sizes (`gcc-11/-Os` and `clang-13/-Os`), nearly half of patch-relevant functions exceed the context limit, and this issue is severer for binaries optimized for efficiency.

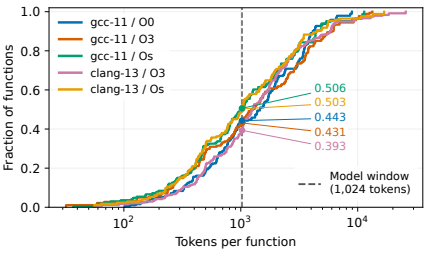


Fig. 1. Accumulated distribution of patch-relevant function tokens of PS^3 dataset.

C6: Missing Usage Context. An isolated function body does not capture how the program uses it. Thin wrappers, common parsing idioms, and repeated utility code may look alike locally but call different semantic callees or appear under different callers. Function-level BCSA is the dominant formulation [12], and `InsnAlign` reports that wrapper queries whose callee semantics are invisible account for most failures in its manual analysis [36]. The surrounding caller-callee relations can therefore contain discriminating evidence absent from a single-function representation.

C7: Patch-Insensitive Similarity. BCSA is trained to keep binaries compiled from the same or closely related source near one another despite compiler variation. That invariance conflicts with patch presence testing, where a few changed instructions must determine the verdict. Vulnerable and patched variants consequently remain near twins: `BinXray` identifies this as a source of false positives, Robin observes patched functions being retrieved as vulnerable, and PS^3 reports that direct function-level similarity cannot reliably separate the two variants [38–40]. A high BCSA score therefore supports code localization, but does not itself establish patch status.

2.3 Instruction Alignment

`InsnAlign` introduces the instruction-to-source mapping as supervision for binary code representation learning [36]. Compiler-generated DWARF information maps binary instructions to their originating source lines [10]. Given two builds of the same source function, `InsnAlign` treats

instructions attributed to the same source line as aligned positives and adds a multi-positive InfoNCE [23] objective to the usual function-level contrastive objective. The approach can augment Transformer backbones such as `jTrans` and `CLAP`, improving function retrieval while exposing instruction-level similarity as inspectable evidence. DWARF supplies instruction correspondences during training; the learned encoder does not require target DWARF at inference.

Formally, let $x = (a_1, \ldots, a_n)$ be an assembly instruction sequence whose serialized and tokenized input fits within the encoder's context window. The learned `InsnAlign` encoder maps x to $\mathbf{z}^x$, the pooled embedding of the input sequence, and $\mathbf{E}^x$, the contextual embeddings of its instructions:

$$(\mathbf{z}^x, \mathbf{E}^x) = \mathrm{Enc}_{\mathrm{IA}}(x), \qquad \mathbf{z}^x \in \mathbb{R}^{d_z}, \;\; \mathbf{E}^x = [\mathbf{e}_1^x, \ldots, \mathbf{e}_n^x]^\top \in \mathbb{R}^{n \times d_e}. \tag{1}$$

These fine-grained instruction representations support inspecting patch-relevant correspondences in the target binary. Although they may be sensitive to some patches, the instruction-alignment objective alone does not fully address C4-C7. The study presents patch testing only as a one-CVE feasibility example and leaves complete binary PPT to future work. In contrast, this paper reuses `InsnAlign`'s model, while it focuses on the remaining challenges for real-world, complicated PPT situations.

3 MOTIVATING EXAMPLE AND DESIGN INSIGHTS

We revisit the tcpdump example introduced in §1 to derive four design insights for end-to-end patch presence testing. These insights address the challenges summarized in §2, from locating patch-relevant behavior across compilation changes to determining its patch status.

Example Details. In the `gcc -O0` reference, `ldp_tlv_print` is a standalone patch-relevant function. In the `gcc -O3` target, its logic is inlined into `ldp_print`, whose optimized body also contains PDU and message processing. The source call chain passes through `ldp_pdu_print`; optimization absorbs these nested calls into the containing function. Fig. 2 illustrates the resulting mismatch between the reference function and an interior region of the target. Function names are shown for explanation; target symbols are not required by the analysis.

```
1  <ldp_print>:
2      ...        ; PDU and message processing
3      ...                          Possible encoder cutoff
4  .tlv_window:
5      ...
6      cmp   captured_bytes, 12
7      jb    .trunc
8      cmp   tlv_bytes, 12
9      jb    .bad_tlv
10     ...
11 .host_continues:
12     ...                          Inlined patch code
13     ret
```

Fig. 2. Simplified patched assembly for CVE-2018-14461. The vulnerable version uses 8 instead of 12 in both bounds checks. Symbolic operands and region labels are preserved for readability; they are stripped in the actual analysis.

Insight 1: Search Within Functions Using Code Windows (C3–C4). Inlining preserves the TLV semantics while removing the standalone `ldp_tlv_print` entry. A patch test that requires this standalone entry loses its analysis target, and one-to-one BCSA has no standalone counterpart to match. Comparing the reference callee with all of `ldp_print` also mixes the desired TLV semantics with substantial caller logic. The relevant correspondence is between a reference function and code *within* a target function. To address this, *retrieval should search code windows inside recovered functions.* `PatchAlign` therefore slices CFGs into windows and searches their embeddings. A window containing relevant TLV logic can nominate `ldp_print` for further analysis.

Insight 2: Preserve Complete Coverage with Instruction Embeddings (C5). Window-based retrieval must also ensure that the relevant region receives a representation. The optimized `ldp_print` is a nontrivial parser. With a finite-context encoder such as `CLAP` [31], truncating its serialized body can omit the inlined patch code; those instructions then contribute nothing to its embedding.

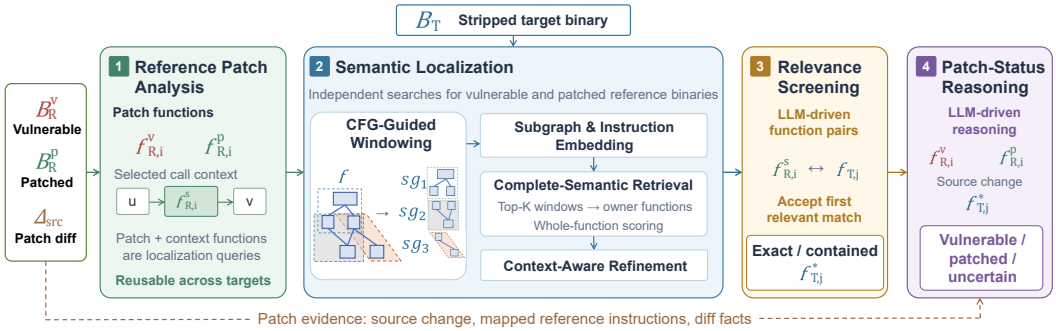


Fig. 3. Workflow of PatchAlign.

Selecting only an entry window would retain this failure. *Every instruction must participate in representation and matching, regardless of the containing function's size.* PatchAlign encodes windows covering the entire function with InsnAlign [36], then scores candidates using instruction embeddings reconstructed across these windows.

Insight 3: Enrich Retrieval with Usage Context (C6). Complete instruction coverage does not make every local code fragment distinctive. Length checks and field loads recur across parsers; in the example, the surrounding LDP PDU and message processing gives the TLV logic additional meaning. More generally, wrappers and utility functions may be ambiguous in isolation but distinctive through their callers and callees. *The usage context of patch-relevant code supplies semantic evidence that its body alone may lack.* PatchAlign retrieves matches for each reference patch function and its caller–callee context, then refines candidate rankings using their semantic and structural consistency while accommodating inlining.

Insight 4: Reason About Patch-Specific Behavior (C7). Even after the correct code is located, the vulnerable and patched TLV parsers share almost all of their behavior. High function-level similarity to either reference cannot establish whether both corrected 12-byte checks guard the relevant field accesses on the FT Session path. Therefore, *patch status requires reasoning about the behavioral change at the relevant site.*

4 SYSTEM DESIGN

PatchAlign first narrows a stripped binary to plausible patch-relevant code, then uses an LLM for relevance screening and patch reasoning. As shown in Fig. 3, PatchAlign takes vulnerable and patched reference binaries, B_R^v and B_R^p, the source change Δ_{src}, and a stripped target binary B_T. It locates patch-relevant code and determines its patch status through four stages.

Reference Patch Analysis (§4.1) extracts patch functions, caller-callee contexts, and patch evidence for reuse across targets. Semantic Localization (§4.2) uses instruction similarity and call context to locate candidate target functions, searching independently from both reference builds. Relevance Screening (§4.3) uses an LLM to reject unrelated candidates. Finally, Patch-Status Reasoning (§4.4) uses an LLM to compare the target code with the source change and both reference states, returning "vulnerable", "patched", or "uncertain".

4.1 Reference Patch Analysis

Reference Patch Analysis turns the source patch and the two reference builds into a localization specification, paired vulnerable and patched functions, one-sided functions, and an analysis plan for distinguishing the paired functions. Its inputs are the vulnerable and patched reference-binary

representations, B_R^v and B_R^p, their instruction-to-source mappings and function symbols, and the sets of source lines removed and added by the patch. Its output is target-independent and can therefore be reused for every target binary tested against the same reference pair.

Recovering Reference Patch Functions. The analysis first finds functions whose source mappings touch a changed line. Exact function symbols pair candidates across the two builds. Because binary PPT is for detecting known vulnerabilities, particularly for vulnerabilities of reused open-source components [9, 40–42], we can build the vulnerable and patched binaries using identical configuration. Similar to PS^3 and `React`, we build the reference binaries with `gcc -O0`. A paired function is retained only when its normalized assembly differs: local labels and hexadecimal immediates are normalized before sequence comparison, and purely cosmetic differences are discarded. Conservative changes in resolved internal call targets provide additional patch evidence when call-edge data is available. A candidate found on only one side is retained rather than being silently dropped. We call each retained patch-relevant function in one reference build a *reference patch function*. Corresponding reference patch functions may be paired across the two builds, while unmatched functions are vulnerable-only or patched-only. Thus, the analysis does not assume that a patch-relevant function survives compilation as a one-to-one function in both references.

Constructing Context Group. For each reference patch function, the analysis builds a directed call graph from resolved internal calls and terminal tail transfers. It then selects a neighborhood centered on that function with bounded caller and callee depths. The selector favors nearby nodes with useful structural evidence, caps high-fanout expansion, suppresses hubs, and enforces a configurable node budget. The production setting explores two caller levels and one callee level. Indirect calls whose destinations cannot be resolved are not invented as edges.

Generating the Plan for Paired Functions. For reference patch functions paired across the two builds based on their function names, an LLM analyzes the source change and the vulnerable and patched functions to identify the behavior that distinguishes them. A second LLM call turns this interpretation into an analysis plan for recognizing these distinctions in other stripped target code. The plan specifies the observations supporting each patch-status label for the paired functions. The interpretation and plan are generated once per CVE and reference-binary pair and reused across target binaries in Patch-Status Reasoning (§4.4). In contrast, one-sided reference functions need no plan to distinguish two versions. When Relevance Screening (§4.3) accepts a target match to a vulnerable-only or patched-only reference function, that function's presence directly provides a "vulnerable" or "patched" vote, respectively.

We write $f_{R,i}^s$ for a reference function used as a localization query, where $s \in \{v, p\}$ identifies the reference build and i identifies the function. The identifiers cover both reference patch functions and the context functions; a separate role label distinguishes them. Some reference patch functions are paired across the two builds, while others are one-sided.

4.2 Semantic Localization

Semantic Localization performs high-recall search with the encoder from `InsnAlign` [36], which produces a pooled representation for coarse retrieval and a contextual representation for every instruction used in fine scoring. For each reference patch function $f_{R,i}^s$ and the set of functions of stripped target binary $\mathcal{F}_T$, semantic localization retrieves a set of candidate target functions $\mathcal{C}(f_{R,i}^s) \subset \mathcal{F}_T$.

4.2.1 ***CFG-Guided Windowing.*** Binary functions can exceed an encoder's token budget. Truncation may omit patch code, while linear slicing can separate branches from the control-flow context needed for high-quality embeddings [8, 31, 35]. We therefore slice each function's control-flow

graph (CFG) into overlapping subgraphs that fit the budget. We denote these windows by $\mathcal{R}(f)$, with $sg^{s}_{R,i,k}$ and $sg_{T,j,t}$ denoting windows of reference function $f^{s}_{R,i}$ and target function $f_{T,j}$, respectively. Window indices are local to each function and imply no correspondence across reference builds or target functions.

For a CFG $G = (V, E)$ with entry block v_0, let $c(v)$ be the tokenizer's token count for basic block v and T the encoder's token budget. Each window selects a block set W satisfying $\sum_{v \in W} c(v) \le T$ and yields an induced subgraph $H = G[W]$ in $\mathcal{R}(f)$. Algorithm 1 constructs windows using three operations, distinguishing primary blocks, which advance coverage tracked by U, from context blocks, which allow overlap between slices.

Choosing a Seed. ChooseSeed selects v_0 if uncovered; otherwise, it selects the lowest-address block in $V \setminus U$. Each seed starts a new primary list, guaranteeing progress. A block exceeding T is split linearly into slices of at most $T/2$ tokens until fully covered.

Algorithm 1 CFG-guided windowing

Require: CFG $G = (V, E)$; entry v_0; token costs c; token budget T; predecessor depth p
Ensure: Ordered slice collection $\mathcal{S}$
1: $U \leftarrow \emptyset$; $\mathcal{S} \leftarrow [\,]$ ▷ primary coverage and output slices
2: **while** $U \neq V$ **do**
3: $s \leftarrow$ ChooseSeed(V, U, v_0)
4: $C, \tau \leftarrow$ BackwardContext(G, s, c, T, p)
5: $P \leftarrow$ ForwardPrimary$(G, s, U, C, c, T - \tau)$
6: $W \leftarrow \mathrm{set}(P) \cup C$; $H \leftarrow G[W]$
7: $B \leftarrow \{u \in W : \mathrm{succ}(u) \cap W = \emptyset \wedge u$ ends in a jump or branch$\}$
8: $\mathcal{S}.\mathrm{append}(H, s, \mathrm{set}(P), C, B)$
9: $U \leftarrow U \cup \mathrm{set}(P)$ ▷ update primary coverage
10: **return** $\mathcal{S}$

Collecting Backward Node. BackwardContext searches the reversed CFG from the seed to depth p using breadth-first search (BFS), considering predecessors in address order at each depth. It adds a predecessor to C if it fits within T alongside the seed and selected context; otherwise, it skips that block and considers others. The operation returns C and its token count τ (Line 4). Context blocks consume budget without updating U.

Growing Primary Coverage. ForwardPrimary performs BFS from the seed in stored successor order, skipping blocks in U or C. It appends visited blocks to the ordered primary list P until the next block would exceed the remaining budget $T - \tau$ (Line 5). The fixed traversal order makes slicing reproducible for a given serialized CFG.

With the collected blocks W, we extract the subgraph H (Line 6) and record conditional-branch blocks with no successor in the slice as cut boundaries B (Line 7).

Finally, we serialize each slice's blocks in increasing address order and rebase internal branch targets to local instruction indices, preserving resolvable control flow and recording cut boundaries. Encoders that support cut markers can append an unknown pseudo-instruction at these boundaries; our model embeds only real instructions and produces one embedding per slice for coarse retrieval.

4.2.2 ***Complete-Coverage Retrieval.*** Algorithm 2 formalizes complete-coverage candidate retrieval. It first embeds each function from its CFG subgraphs and then uses all resulting subgraph embeddings for retrieval across the target binary.

While embedding a function f, CFG slicing represents it as a set of subgraphs $\mathcal{R}(f)$. For each subgraph sg, EmbedSubgraph directly reuses the learned `InsnAlign` encoder defined in Eq. (1). Here, Serialize(sg) produces the instruction sequence using the serialization described in §4.2.1. The encoder returns the pooled subgraph embedding $\mathbf{z}^{sg}$ and the contextual instruction embeddings $\mathbf{E}^{sg}$; we index the latter by the original instruction identifiers so that $\mathbf{E}^{sg}[a] = \mathbf{e}^{sg}_{a}$ for each instruction a in sg. We denote the embeddings of all subgraphs of f by $\mathcal{Z}(f)$. Because these subgraphs jointly cover the entire function, $\mathcal{Z}(f)$ allows retrieval to consider the function's complete semantics even when the function exceeds the encoder's input budget. The subgraphs can overlap, so an instruction may

Algorithm 2 Complete-Coverage Retrieval

Require: A reference query $f^s_{R,i}$, the target functions $\mathcal{F}_T$, the `InsnAlign` encoder Enc_{IA} from Eq. (1), and the number K of target subgraphs to retrieve per reference subgraph
Ensure: The candidate set $\mathcal{C}(f^s_{R,i})$ of target functions

function EmbedSubgraph(sg)
  $x \leftarrow \text{Serialize}(sg)$
  return $\text{Enc}_{\text{IA}}(x)$ ▷ $(\mathbf{z}^{sg}, \mathbf{E}^{sg})$; Eq. (1)
function EmbedFunction(f) ▷ f contains n instructions
  $\mathcal{R}(f) \leftarrow$ SliceCFG(f)
  $\mathcal{Z} \leftarrow [\,]$
  $\mathcal{H} \leftarrow [\emptyset, \ldots, \emptyset]$ ▷ One set for each instruction in f
  for all $sg_k \in \mathcal{R}(f)$ **do**
    $\mathbf{z}_k, \mathbf{E}_k \leftarrow$ EmbedSubgraph(sg_k)
    $\mathcal{Z}$.append($\mathbf{z}_k$)
    for all $a \in \text{instructions}(sg_k)$ **do**
      $\mathcal{H}[a] \leftarrow \mathcal{H}[a] \cup \{\mathbf{E}_k[a]\}$
  for all $a \in \{1, 2, \ldots, n\}$ **do**
    $\mathbf{e}_a \leftarrow \frac{1}{|\mathcal{H}[a]|} \sum_{\mathbf{e} \in \mathcal{H}[a]} \mathbf{e}$ ▷ Average across overlapping subgraphs
  return $\mathcal{Z}, \mathbf{E} = (\mathbf{e}_1, \ldots, \mathbf{e}_n)$
function CoarseRetrieval($f^s_{R,i}, \mathcal{F}_T, K$)
  $\mathcal{Z}^s_{R,i}, \mathbf{E}^s_{R,i} \leftarrow$ EmbedFunction($f^s_{R,i}$)
  $m \leftarrow |\mathcal{Z}^s_{R,i}|$
  $\mathcal{Z}^{\text{all}} \leftarrow [\,]$ ▷ All target subgraph embeddings
  Owner $\leftarrow [\,]$ ▷ Owner function aligned with each target embedding
  for all $f_{T,j} \in \mathcal{F}_T$ **do**
    $\mathcal{Z}_{T,j}, \mathbf{E}_{T,j} \leftarrow$ EmbedFunction($f_{T,j}$)
    for all $\mathbf{z} \in \mathcal{Z}_{T,j}$ **do**
      $\mathcal{Z}^{\text{all}}$.append($\mathbf{z}$)
      Owner.append($f_{T,j}$)
  $\mathbf{M} \leftarrow \text{cosine}(\mathcal{Z}^s_{R,i}, \mathcal{Z}^{\text{all}})$ ▷ $\mathbf{M}$ has shape $m \times |\mathcal{Z}^{\text{all}}|$
  $\mathcal{C}(f^s_{R,i}) \leftarrow \emptyset$
  for all $k \in \{1, 2, \ldots, m\}$ **do**
    $\mathcal{Q}_k \leftarrow \text{TopKIndices}(\mathbf{M}[k], K)$ ▷ Indices of the top-K target subgraphs for reference subgraph
    for all $q \in \mathcal{Q}_k$ **do**
      $\mathcal{C}(f^s_{R,i}) \leftarrow \mathcal{C}(f^s_{R,i}) \cup \{\textbf{Owner}[q]\}$ ▷ Add the owner function of the retrieved subgraph
  return $\mathcal{C}(f^s_{R,i})$

receive multiple contextual embeddings. We reconstruct a single contextual embedding for each instruction in the function by averaging all of its subgraph-specific embeddings. If $\mathcal{R}_a(f) \subseteq \mathcal{R}(f)$ is the set of subgraphs containing instruction a, Line 14 performs this averaging over all subgraphs in $\mathcal{R}_a(f)$. For a specific function, we write $\mathcal{Z}^s_{R,i} = \mathcal{Z}(f^s_{R,i})$ and $\mathcal{Z}_{T,j} = \mathcal{Z}(f_{T,j})$, with reconstructed instruction embeddings $\mathbf{E}^s_{R,i}$ and $\mathbf{E}_{T,j}$, respectively.

For the overall retrieval, we apply this embedding procedure to the reference query $f^s_{R,i}$ and every target function $f_{T,j} \in \mathcal{F}_T$. For each reference window $sg^s_{R,i,k}$, we retrieve the K target windows $sg_{T,j,\ell}$ whose embeddings have the highest cosine similarity. A target function $f_{T,j}$ becomes a candidate if it owns at least one retrieved window $sg_{T,j,\ell}$. Lines 29–31 select the top-K target subgraphs and insert their owners into $\mathcal{C}(f^s_{R,i})$. This set union removes duplicate functions and allows different reference subgraphs to retrieve candidates independently.

Whole-Function Scoring. After candidate retrieval, we rank only the functions in $\mathcal{C}(f^s_{R,i})$ using their whole-function instruction embeddings. For a candidate target function $f_{T,j}$, let I_R and I_T be the instruction sets of $f^s_{R,i}$ and $f_{T,j}$, respectively. Let $S_{ab} = \cos(\mathbf{E}^s_{R,i}[a], \mathbf{E}_{T,j}[b])$ be the cosine similarity between reference instruction $a \in I_R$ and target instruction $b \in I_T$. Their symmetric

Mean Alignment Score (MAS) [36] is

$$\mathrm{MAS}(f^s_{R,i}, f_{T,j}) = \frac{1}{2}\left(\frac{1}{|I_R|}\sum_{a\in I_R}\max_{b\in I_T} S_{ab} + \frac{1}{|I_T|}\sum_{b\in I_T}\max_{a\in I_R} S_{ab}\right). \tag{2}$$

Candidates are sorted in descending MAS order. Thus, pooled subgraph cosine similarity determines candidate membership, while whole-function MAS determines the final retrieval ranking.

4.2.3 ***Context-Aware Refinement.*** Whole-function similarity leaves two ambiguities: absolute scores are not directly comparable across reference functions, and optimization can inline a reference function into another function, removing its standalone counterpart. Our key insight is that a plausible target location should both have strong *relative* semantic support and explain the surrounding caller-callee relations, allowing changes in function boundaries. We combine per-reference score normalization with an inlining-aware graph edit search. For each reference patch function $f^s_{R,i}$ (not the context functions) and its retained candidate $f_{T,j} \in \mathcal{C}(f^s_{R,i})$, the lowest cost of their context graph edits reflects the relative semantic support and determines the rank of $f_{T,j}$.

Relative Semantic Evidence. For this subsection, write $f = f^s_{R,i}$ for a reference query and $t = f_{T,j}$ for a retained target candidate. Let $s(f,t)$ be their MAS score, and let $s^+(f)$ and $s^-(f)$ be the greatest and least retained scores. When these differ, we define

$$\mathrm{rel}(f,t) = \frac{s(f,t) - s^-(f)}{s^+(f) - s^-(f)}, \qquad D(f,t) = 1 - \mathrm{rel}(f,t). \tag{3}$$

The best and least similar retained candidates therefore have mismatch costs zero and one, respectively. This expresses support within each reference ranking, not a probability of correctness.

Inlining-Aware Context Edits. We adapt graph edit distance (GED) [1] to match a small reference call context inside the target call graph [22, 26]. Alongside node substitution, edge substitution, and edge deletion, we add an operation, `merge_edge_to_node`. For an observed reference call $u \to v$, this operation maps both endpoints to one target t: contracting callee v into caller u models inlining. The removed node's other call relations remain to be matched, so a contraction changes function boundaries without discarding context. Edit costs reflect the evidence supporting each transformation. Node substitution uses $D(f,t)$, making strong relative semantic matches cheap. Callee inlining costs

$$C_{\mathrm{inl}}(v \Downarrow u, t) = \sigma(v) D(v,t), \tag{4}$$

where $\sigma(v)$ is the fraction of profiled instructions with semantic content. Thus a merge is inexpensive when the target strongly matches the removed callee or that callee performs little independent computation.

Edge costs measure call connectivity: a direct target call costs zero, longer directed paths cost more, and absence of a path within the search horizon incurs edge deletion. Reference nodes cannot be arbitrarily deleted; extra target structure is free because the reference context is sought *inside* the target graph.

Context-Based Ranking. For each reference patch function $f^s_{R,i}$, we reuse the context group constructed during Reference Patch Analysis (§4.1) and form target candidate locations from its members' independently retrieved matches and bounded caller-callee expansion in the target call graph. This lets context recover a location that the patch function alone ranks poorly. For each candidate, we fix the patch function's mapping and use bounded beam search to find a low-cost context explanation. We rank candidates directly by the lowest mean edit cost found, normalized by the number of reference nodes and edges, and pass this ranking to Relevance Screening. The original top-MAS candidate is evaluated under the same edit objective as the other candidates.

4.3 Relevance Screening

Context-aware refinement produces a ranked list of candidate target functions $f_{T,j}$ for each reference patch function $f^s_{R,i}$. A high rank, however, does not prove that a candidate implements the same high-level logic. For example, shared utility code can produce both strong instruction similarity and a plausible local call pattern. Relevance Screening therefore asks an LLM to determine the relationship between each reference patch function and only its first K_R ranked candidate target functions.

Each prompt contains exactly one reference-target pair $(f^s_{R,i}, f_{T,j})$: the complete code (assembly or decompiled pseudocode) of the reference patch function from its vulnerable or patched reference build, and the complete code of the candidate target function. It does not expose semantic similarities or structural edit costs. The candidate's own symbol and self-references are replaced with an address-derived anonymous name, and target source mappings are removed; meaningful names of other called functions may remain as assembly evidence. The LLM classifies the relation as "exact", "contained", or "wrong". "exact" means the candidate implements the same high-level function. "contained" means the reference patch function's semantics occur within a different target function, as can result from inlining, function merging, or function specialization. "wrong" means the candidate neither implements the same high-level function nor contains the reference patch function's semantics.

Candidates are checked in structural rank order. The first candidate classified as "exact" or "contained" becomes the accepted target function $f^\star_{T,j}$ for that reference patch function. The star marks acceptance and does not change the target function's identity. Lower-ranked candidates are recorded as skipped without another LLM call. This choice saves token consumption and it is still reliable; our experiments in §5.4 demonstrates that advanced LLMs can classify irrelevant code with nearly perfect accuracy. This early-stop policy concentrates generative cost on ambiguous high-ranked pairs while preserving an explicit "wrong" outcome for incidental retrieval hits.

Function Status and Handoff. For a one-sided reference patch function, an accepted "exact" or "contained" match establishes that its behavior is present in the target. The accepted function therefore contributes a "vulnerable" vote if the reference function is vulnerable-only, or a "patched" vote if it is patched-only. It needs no further comparison in Patch-Status Reasoning.

4.4 Patch-Status Reasoning

This stage determines the status of accepted target functions associated with paired reference patch functions, then aggregates function votes into the target binary's verdict. One-sided functions already receive their status votes during Relevance Screening (§4.3).

Reasoning for Paired Functions. For an accepted target function $f^\star_{T,j}$ associated with paired reference patch functions $f^v_{R,i}$ and $f^p_{R,i}$, the LLM receives:

- the cached patch interpretation and analysis plan produced by Reference Patch Analysis (§4.1);
- the complete code of both the vulnerable reference function $f^v_{R,i}$ and the patched reference function $f^p_{R,i}$; and
- the complete anonymized code of $f^\star_{T,j}$, represented as assembly or decompiled pseudocode.

Following the plan, the LLM first identifies the patch-relevant behavior within the accepted target function, then checks it against the observations that distinguish the two reference functions. It returns a function-level label of "vulnerable", "patched", or "uncertain", with a concise explanation grounded in the target code and both reference functions. The same plan is reused whenever this reference pair is tested against another target; it is not regenerated during target reasoning.

Aggregating Function Votes. `PatchAlign` collects the votes from all accepted patch-relevant functions: the status votes assigned to one-sided functions during Relevance Screening and the

Table 1. Statistics of PS^3-Balanced. Each CVE contributes one vulnerable and one patched `gcc-11 -O0` reference, and four vulnerable and four patched targets built with GCC and Clang at `-O3` and `-Os`.

Project	CVEs	Ref. Binaries	Target Binaries		Total Binaries
			Vulnerable	Patched	
FFmpeg	26	52	104	104	260
OpenSSL	23	46	92	92	230
tcpdump	11	22	44	44	110
libxml2	2	4	8	8	20
Total	**62**	**124**	**248**	**248**	**620**

labels produced by reasoning for paired functions. Let V, P, and U denote "vulnerable", "patched", and "uncertain", respectively, and let $S \subseteq \{V, P, U\}$ be the set of observed vote labels. The target binary is classified as "vulnerable" if $V \in S$ and $P \notin S$, and as "patched" if $P \in S$ and $V \notin S$. PatchAlign returns "uncertain" if both V and P occur in S, or if $S = \{U\}$. When no functions are accepted ($S = \emptyset$), PatchAlign treats the binary as "not affected". This outcome is recorded as **MISSING** and assigned to the negative class alongside "patched" outcomes in the evaluation.

5 EVALUATION

We organize the evaluation around the following research questions.

- **RQ1 (End-to-end effectiveness):** How accurately does PatchAlign identify vulnerable and patched target binaries compared with React and PS^3?
- **RQ2 (Localization):** How effectively do our enhancements contribute to the localization?
- **RQ3 (Relevance decision):** How reliably does the LLM classify candidate relationships as "exact", "contained", or "wrong"?
- **RQ4 (Efficiency):** What time, model-query, token, and monetary cost does PatchAlign incur in its offline and online analysis stages?

5.1 Experimental Setup

The PS^3-Balanced Benchmark. We rebuild the benchmark released with PS^3 [40] to remove a confounding factor. Its CVE-binary observations are distributed unevenly across CVEs. Consequently, a micro-average can be dominated by CVEs with long release histories or skewed class distributions. We call our reconstructed benchmark PS^3-Balanced. It retains all 62 CVEs from the four original projects and builds the vulnerable and patched source revisions under each configuration. We use only `gcc-11 -O0` binaries as references; targets are built with `gcc-11` and `clang-13` at `-O3` and `-Os`. Thus, each reference-target setting contains exactly 62 vulnerable and 62 patched targets. As summarized in Table 1, PS^3-Balanced contains 124 reference binaries and 496 target binaries. We retain paired stripped and unstripped artifacts, using the unstripped copies only to construct and audit ground truth.

Baselines. We compare PatchAlign with the following three baselines:

- **PS^3.** PS^3 uses symbolic emulation to extract semantic signatures from the reference binaries and compares them with a specified target function [40]. Our adapter additionally recovers an inlined reference function through its DWARF-identified caller; we therefore label this strengthened baseline PS^3 *with inline recovery*.
- **React (symbol).** This configuration uses function symbols to locate the target function, lifts the reference and target functions to an intermediate representation, and uses refined semantic features and an SMT solver to determine patch status [41]. This is the official React artifact released with the paper.

- **React (Sem-Loc).** This diagnostic configuration replaces React's symbol-based target lookup with our Semantic Localization (§4.2) results while retaining React's reference processing and patch-presence testing. We select React with our localization front end because it is publicly available and presents a better performance than prior works [40, 41].

Implementation. The Semantic Localization (§4.2) of PatchAlign uses the InsnAlign-enhanced CLAP checkpoint released by InsnAlign [36], a top-$K = 50$ target-window retrieval shortlist for each reference window, and a 1,024-token window budget (i.e., the same setting used by CEBin [32]). For each reference patch function, we collect its context group with bounded caller and callee edges of length of two, as previous studies [13, 33] have demonstrated this setting is sufficient for detecting inlined semantics. The Context-Aware Refinement search (§4.2.3) uses deterministic bounded beam search to approximate the minimum directed graph edit cost with beam width 32.

We use the deepseek-v4-flash-0731 model with temperature 0, max reasoning effort, and 1M context limit for relevance screening and patch-status reasoning. Relevance Screening iterates over at most 10 candidates per reference-side function. Patch-Status Reasoning analyzes accepted "exact" or "contained" matches to paired reference functions.

5.2 RQ1: End-to-End Effectiveness

Table 2 compares the four settings with a gcc-11 -O0 reference. Each setting contains the same 62 vulnerable and 62 patched targets.

Metrics. We treat *vulnerable* as the positive class and report *T*rue and *F*alse *P*osiives and *N*egatives (i.e., TP, FP, TN, and FN). Because abstention is operationally meaningful, we also report the number of **U**ncertain cases. Table 2 presents the **P**recision, **R**ecall, **F1**, and accuracy (**Acc.**) of all methods; those metrics are commonly used for measuring the performance of binary PPT solutions [15, 40–42]. They are computed by $\mathbf{P} = \frac{TP}{TP+FP}$, $\mathbf{R} = \frac{TP}{TP+FN}$, $\mathbf{F1} = \frac{2\times P\times R}{P+R}$, and $\mathbf{Acc} = \frac{TP+TN}{TP+FP+TN+FN}$, respectively. We also provide non-missing uncertain count, the **MISSING** count, and selected accuracy (**Sel. Acc.**) for diagnosis.

Given the potential catastrophic effect of missing a known vulnerability, similar to how previous solutions, which treats uncertain cases as vulnerable (positive), we adopt a similar policy [40–42]: All **U**ncertain cases are treated as vulnerable, contributing TP and FP. In contrast, **MISSING** denotes that the solution does not find the patch-relevant code in the stripped binary, indicating the target is considered not being affected by the vulnerability. This can be caused by missing symbols, semantic localization failures, and relevance screening mistakes. MISSING cases contribute to negatives (i.e., TN and FN), tending to reduce the recall.

Symbol-Based Patch-Relevant Function Identification. For PS^3 and React (symbol), the MISSING cases are mainly caused by missing symbols in the target binary, which is a common scenario in real-world binaries due to compilation optimizations like function inlining. Note that we still provide unstripped binaries for them to use symbols for localization, and stripping binaries can further increase MISSING cases in practice. In contrast, PatchAlign does not rely on symbols and can localize patch-relevant code even in stripped binaries, resulting in limited MISSING cases across all settings.

The small MISSING differences between PS^3 and React (symbols) come from two artifact-specific cases. For CVE-2019-13312, PS^3 cannot construct a reference signature for the ambiguous static name encode_init and labels both target states MISSING under all four configurations. For CVE-2018-16228, React (symbols) failed to lift the LLVM IR for particular patch-relevant functions.

For official PS^3 and React, the number of MISSING cases are not negligible. Given 496 target binaries, over 25% binaries do not contain the expected function symbols. As a result, the accuracy decreases significantly when treating those MISSING cases as negatives. Moreover, our experiments

Table 2. Results for PS^3 dataset. Vulnerable is positive, and patched is negative. **U** is # non-missing uncertain outcomes. **MISSING** denotes # targets that tool fails to localize the patch-relevant code. **Sel. Acc.** is the decision accuracy excluding U and MISSING cases.

Setting	Method	U	*TP*	*FP*	*TN*	*FN*	MISSING	P	R	F1	Sel. Acc.	Acc.
GCC -O0→GCC -O3	PS^3+inline	0	42	15	47	20	39	73.68	67.74	70.59	81.18	71.77
	React (symbols)	0	44	12	50	18	37	78.57	70.97	74.58	86.21	75.81
	React (Sem-Loc)	9	60	27	35	2	3	68.97	**96.77**	80.54	79.46	76.61
	PatchAlign	8	58	4	58	4	3	93.55	93.55	93.55	96.46	93.55
	PatchAlign (decompile)	3	60	2	60	2	6	**96.77**	**96.77**	**96.77**	**100.00**	**96.77**
GCC -O0→Clang -O3	PS^3+inline	0	44	22	40	18	31	66.67	70.97	68.75	74.19	67.74
	React (symbols)	0	43	13	49	19	30	76.79	69.35	72.88	81.91	74.19
	React (Sem-Loc)	10	54	23	39	8	2	70.13	87.10	77.70	77.68	75.00
	PatchAlign	5	57	3	59	5	2	95.00	91.94	93.44	94.87	93.55
	PatchAlign (decompile)	2	61	1	61	1	2	**98.39**	**98.39**	**98.39**	**100.00**	**98.39**
GCC -O0→GCC -Os	PS^3+inline	0	45	19	43	17	32	70.31	72.58	71.43	78.26	70.97
	React (symbols)	0	44	12	50	18	31	78.57	70.97	74.58	83.87	75.81
	React (Sem-Loc)	12	58	23	39	4	2	71.60	93.55	81.12	82.73	78.23
	PatchAlign	5	60	2	60	2	2	96.77	**96.77**	**96.77**	98.29	**96.77**
	PatchAlign (decompile)	1	58	1	61	4	8	**98.31**	93.55	95.87	**100.00**	95.97
GCC -O0→Clang -Os	PS^3+inline	0	47	23	39	15	28	67.14	75.81	71.21	75.00	69.35
	React (symbols)	0	46	18	44	16	27	71.88	74.19	73.02	78.35	72.58
	React (Sem-Loc)	11	57	27	35	5	2	67.86	91.94	78.08	77.48	74.19
	PatchAlign	9	57	5	57	5	2	91.94	91.94	91.94	95.58	91.94
	PatchAlign (decompile)	2	60	1	61	2	4	**98.36**	**96.77**	**97.56**	**100.00**	**97.58**
Overall	PS^3+inline	0	178	79	169	70	130	69.26	71.77	70.50	77.05	69.96
	React (symbols)	0	177	55	193	71	125	76.29	71.37	73.75	82.48	74.60
	React (Sem-Loc)	42	229	100	148	19	9	69.60	92.34	79.38	79.33	76.01
	PatchAlign	27	232	14	234	16	9	94.31	93.55	93.93	96.30	93.95
	PatchAlign (decompile)	8	239	5	243	9	20	**97.95**	**96.37**	**97.15**	**100.00**	**97.18**

of React (Sem-Loc) demonstrate that the missing symbols is the superfacial outcome, forcing existing patch-status classifier to analyze the patch-relevant code does not close this gap.

Semantic-Localization-Assisted React. React (Sem-Loc) uses our Semantic Localization and Relevance Screening results for selecting binary functions in the target binary, but retains React's reference analysis and classifier. It reduces MISSING from 125 to 9 and increases recall and F1 compared with React (symbols). The evaluation of React also demonstrates that Semantic Localization is effective for patch-relevant code retrieval, but the final end-to-end performance is still limited by the classifier's ability to distinguish patched and vulnerable states. The Sel. Acc. of React (Sem-Loc) of all settings also decreases considerably (from 86.21% to 79.46% of `gcc -O0→gcc -O3`), indicating that aggressive optimization changes can largely affect the classifier's ability to distinguish patched and vulnerable states, even when the patch-relevant code is successfully localized.

Results of PatchAlign. Overall, PatchAlign reaches 93.95% accuracy and 93.93% F1, significantly exceeding the best corresponding baseline results. When we use decompiled pseudocode for relevance screening and decision-making, PatchAlign achieves 97.18% accuracy and 97.15% F1, which is the best among all settings. Compared with the baselines, PatchAlign's main advantage is precision: it reduces the aggregate false positives (FP) from over hundred for the baselines to 14 and 5, while retaining over 96% recall. Together with small MISSING, these results show that PatchAlign provides a substantially better balance between detecting vulnerable binaries and avoiding false alarms across compiler and optimization changes.

5.3 RQ2: Patch-Relevant Code Localization

We measure localization performance using Recall@k, the fraction of queries for which at least one correct target function appears among the top-k candidates. This metric is widely used in information retrieval and binary code similarity analysis (BCSA) [12, 36].

Patch-Relevant Ground Truth. Conventional BCSA evaluations generally define a correct match by source-function identity, often recovered from symbols or function names. Our ground truth instead captures the target function that implements the patch-relevant behavior. We treat a target function as correct when it is either an exact counterpart or a different function containing the reference patch function's semantics. Inlining is one way this containment can arise. For instance, the patch-relevant function of CVE-2018-14461 is `ldp_tlv_print` in the reference binaries, which is inlined into `ldp_print` in the target; we therefore count `ldp_print` as a correct match. We construct the ground-truth cache from the patch-analysis inventory and original unstripped ELF/DWARF binaries, following the use of debug information in prior evaluation [13, 14, 36].

Experimental Setup. We evaluate four successive stages: cosine retrieval with linear windows, CFG-guided windowing (§4.2.1), whole-function MAS (§4.2.2), and context-aware refinement (§4.2.3). Following RQ1, we use `gcc -O0` references and the four optimized target settings in Table 2. We compare vulnerable references with vulnerable targets and patched references with patched targets. Across all 62 CVEs, the `gcc -O0` reference binaries provide 192 patch-relevant functions (95 from vulnerable binaries and 97 from patched binaries), yielding 768 (= 192×4) localization queries across the four target settings. The overall results are the pooled performance across all four settings. Following prior BCSA evaluations [8, 19, 31, 33, 35, 36], we assess mean reciprocal rank (MRR) and use the one-sided paired t-tests on per-query reciprocal ranks and report Holm-adjusted p-values (p_{H}) for successive improvements [28, 30]. Large t and small p_{H} reflect significant improvements. MRR uses the first correct candidate's rank in the retrival list, with zero when no correct candidate is present.

Table 3. Overall patch-relevant code-localization performance over 768 queries. Significance tests compare each stage with its predecessor.

Method	Recall@1/5/10	MRR	t_{MRR}	p_{H}
Cosine (linear)	65.23/76.30/79.17	0.7007	–	–
Cosine (subgraph)	70.70/78.78/80.99	0.7419	4.48	4.33×10^{-6}
MAS (§4.2.2)	84.11/89.19/90.23	0.8616	10.47	6.60×10^{-24}
Context (§4.2.3)	87.11/97.27/97.53	0.9146	4.99	3.69×10^{-7}
Context (no-rel)	23.44/65.23/80.21	0.4163	–	–
Context (no-inl)	76.17/85.55/87.50	0.8052	–	–

Results. As shown in Table 3, all three enhancements increase pooled recall and significantly improve pooled MRR. The most notable gains come from MAS, which employs the whole-function semantics for BCSA, demonstrating the importance of high semantics coverage for patch-relevant code localization. The following context-aware refinement considers the call context of each candidate and further improves recall and MRR. Compared with MAS, Recall@10 increase from 90.23% to 97.53%, a high-recall shortlist for Relevance Screening.

Context (no-rel) replaces $\mathrm{rel}(f, t)$ with the absolute MAS score $s(f, t)$ in Eq. 3, reducing Recall@10 to 80.21%. High absolute scores can arise from semantics shared by multiple candidates; normalization rescales their differences within each reference ranking so that context edit costs reflect relative semantic support. Context (no-inl) assigns a prohibitive cost of 10^{12} to `merge_edge_to_node`, reducing Recall@10 to 87.50%. Without this operation, a reference caller and callee must match separate target functions, so inlining can cause correct matches to be penalized as context mismatches despite preserved semantics.

5.4 RQ3: Reliability of Relevance Screening

Relevance screening (§4.3) distinguishes "exact" and "contained" relationships from "wrong" candidates that exhibit incidental similarity in the rankings produced by context-aware refinement

Table 4. Relevance screening results, reported as correct/total (percentage).

Setting	End-to-end				Conditional on valid top-10			
	Exact	Contained	Overall	Irrelevant	Exact	Contained	Overall	Irrelevant
Assembly	526/546 (96.34%)	205/222 (92.34%)	737/768 (95.96%)	356/369 (96.48%)	526/536 (98.13%)	205/213 (96.24%)	737/749 (98.40%)	217/224 (96.88%)
Decompiled C	523/546 (95.79%)	213/222 (95.95%)	743/768 (96.74%)	380/380 (100.00%)	523/536 (97.57%)	213/213 (100.00%)	743/749 (99.20%)	190/190 (100.00%)

(§4.2.3). We evaluate the combined retrieval-and-screening outcome and isolate screening performance on queries with a correct top-10 candidate.

`CI-Detector` [14] is a pairwise binary classifier that distinguishes cross-inlining pairs from non-cross-inlining pairs. We therefore exclude it from RQ2, which evaluates large-scale function localization and requires extreme efficiency [20, 32, 33]. It is also unsuitable for relevance screening, which must distinguish exact matches, semantic-containment matches (e.g., those arising from inlining [13], function merging [25], or function specialization [4]), and irrelevant pairs, while `CI-Detector` places all non-cross-inlining relationships in a single negative class.

Metrics. Using the ground truth from §5.3, Exact requires a relevant target whose symbol matches the reference after removing compiler-added suffixes; Contained requires the reference behavior within a target with a different base symbol. Both measure query success with the correct relation label; queries with an exact target belong only to Exact. Overall accepts either positive label for any relevant target, counting each query once. Irrelevant measures the fraction of evaluated ground-truth irrelevant pairs correctly labeled "irrelevant". End-to-end uses all eligible queries; conditional rates require a corresponding ground-truth candidate in the top-10 (either positive relation for Overall and Irrelevant).

Results. Table 4 shows that decompiled C achieves higher Overall success and is more reliable on Contained and Irrelevant decisions, while assembly retains a small Exact advantage. The Contained improvement holds across all compiler/optimization settings: decompiled C preserves every assembly success and resolves eight additional queries. Its stronger rejection of irrelevant candidates also holds on pairs evaluated by both methods. Since both representations use identical candidate rankings, these advantages arise during screening.

Comparing relations exposes different weaknesses. For assembly, Contained remains harder than Exact even when a true candidate is retrieved. Decompiled C succeeds on every retrieved Contained query, so containment is not its main screening limitation in these experiments. Its Exact shortfall includes false rejections and Exact-to-Contained label swaps. Both methods also sometimes accept a valid containing host before the exact counterpart, preserving Overall success while reducing Exact success.

5.5 RQ4: Efficiency and Cost

Table 5 separates reusable offline analysis from online PPT. Local stages report wall time; LLM stages report summed request elapsed time and recorded input/output tokens, including reasoning tokens.

Implementation and Environment. We measure 62 CVEs and 496 targets under the four `gcc-11/-O0`-reference settings in Table 1, starting from disassembled assembly code. We use Ghidra to produce the pseudocode for our decompilation pipeline. The experiment was conducted on a 16-core AMD 9950X3D2 server with 256 GiB RAM and an NVIDIA GeForce RTX 5090 (32 GiB). The LLM was `deepseek-v4-flash-0731` at temperature zero and max reasoning effort.

Cost Distribution. Because `PatchAlign` (decompile) merely requires the pseudocode of several relevant functions while screening, the cost of decompiling these functions is negligible and omitted. For `PatchAlign` (decompile), Windowing + Retrieval consumes 59.3% of online time, followed by

Table 5. Offline and online cost of PatchAlign over 62 CVEs and 496 targets.

Phase and component	Time				LLM calls	Input tok.	Output tok.
	Total (h)	Mean (s)	p50 (s)	p95 (s)		Total / mean	
Offline: once per CVE and reference pair							
LLM CVE interpretation and plan	1.88	109	97	233	124 / 2	303,918 / 4,902	707,006 / 11,403
Online: once per target binary							
Windowing + Retrieval	154.46	1,121	521	4,182	–	–	–
Context-Aware Refinement	5.19	38	27	117	–	–	–
Relevance Screening (assembly)	91.28	663	322	2,223	3,312 / 6.68	34.26M / 69,078	37.52M / 75,645
Relevance Screening (decompile)	81.17	589	299	2,055	2,385 / 4.81	36.00M / 72,590	34.82M / 70,198
Patch-Status Reasoning (assembly)	27.04	196	136	607	757 / 1.53	8.03M / 16,195	12.28M / 24,767
Patch-Status Reasoning (decompile)	19.59	142	70	516	762 / 1.54	23.40M / 47,170	7.59M / 15,292
Online total (assembly)	277.98	2,018	1,425	4,997	4,069 / 8.20	42.30M / 85,273	49.80M / 100,413
Online total (decompile)	260.42	1,890	1,246	4,793	3,147 / 6.34	59.40M / 119,760	42.40M / 85,490

Relevance Screening (31.2%), Patch-Status Reasoning (7.5%), and Context-Aware Refinement (2.0%). Online costs are right-skewed: each stage's p95 is 6.8× its median. With the same localization cost, the decompiled total is 1,890s per target, assembly-only analysis takes additional 6.77% time due to increased output tokens.

Assembly and Decompiled Pseudocode. Pseudocode-based screening uses 28.0% fewer queries and 11.1% less summed time, but similar input and output token consumption as assembly screening. Its final reasoning uses similar LLM calls, while the number of input tokens is 2.91× as that of assembly-based reasoning; in contrast, the output tokens of using pseudocode is merely 61.8% of assembly-based reasoning. The significantly different distribution of input and output tokens indicates that decompilation introduces additional tokens for LLM to analysis and alleviates the reasoning efforts.

For prices $p_i = \$0.15$ and $p_o = \$0.6$ per million input and output tokens of `deepseek-v4-flash` [6], the recorded offline cost is $0.304p_i + 0.707p_o = \$0.47$; \$0.0076 per CVE on average. Online LLM costs are $42.30p_i + 49.80p_o = \$36.23$ for assembly and $59.40p_i + 42.40p_o = \$34.35$ for pseudocode, i.e., \$0.073 (assembly) and \$0.069 (pseudocode) per target binary per CVE on average.

Comparison with Baselines. Table 6 compares target-side patch-status reasoning costs after reusable preparation. The PS^3 and React timers excludes the disassembly and offline signature generation. React additionally includes the time to lift the assembly of target binaries to IR. Overall, the patch-status reasoning cost of PS^3 and React is nearly negligible compared with the semantic localization cost. PatchAlign's final reasoning costs more than either baseline's local matching: its summed time equals 16.94% of deterministic localization time for assembly and 12.27% for pseudocode. However, given the accuracy rises from 74.80% (React Sem-Loc) to 96.17% (PatchAlign decompile), these costs are justified.

Table 6. Patch-Status Reasoning time costs (s).

Method	Total	Mean	p50	p95	Cost/Search
PatchAlign (assembly)	97,354	196.28	136.46	607.10	16.94%
PatchAlign (decompile)	70,521	142.18	70.47	515.76	12.27%
PS^3+inline (target+decision)	959	1.93	1.25	4.88	0.17%
React (lift+decision)	2,306	4.65	2.74	15.21	0.40%

5.6 Case Study: Misleading Decompiled Pseudocode

Correct Localization. For CVE-2021-23841, the correct function ranks first after semantic localization on both reference sides. The affected OpenSSL function, `X509_issuer_and_serial_hash`, is located in a patched `gcc-11 -O3` target using `gcc-11 -O0` references. The fix checks the result of `X509_NAME_oneline` for null before `strlen` [3]; this guard is visible in Listing 1 (Line 5).

```
1  /* Patched reference source. */                          | /* Decompiled patched target. */
2  f = X509_NAME_oneline(                                   | __s = FUN_1a0350(
3      a->cert_info.issuer,                                 |     *(X509_NAME **)&(a->ex_data).dummy,
4      NULL, 0);                                            |     (char *)0x0, 0);
5  if (f == NULL)  /* patch-added guard */                  | if (__s != (char *)0x0) {
6      goto err;                                            |     ...
7  ...                                                      |     if (iVar1 != 0) {
8                                                           |         ...
9                                                           |         if (iVar1 != 0) {
10                                                          |             ...
11 if (!EVP_DigestUpdate(ctx,                               |             iVar1 = FUN_11e8b0(ctx,
12     (unsigned char *)a->cert_info.serialNumber.data,     |                 a->signature,
13     (unsigned long)a->cert_info.serialNumber.length))    |                 (long)*(int *)&a->sig_alg);
14     goto err;                                            |             if (iVar1 != 0) {
15 ...                                                      |                 ...
```

Listing 1. Lines 11–13 show corresponding calls and arguments. Meaningful target types and field names come from Ghidra's library type archive and survive function-name anonymization. In lines 12–13, Ghidra **incorrectly assigns** the `signature` and `sig_alg` to the memory locations accessed by the blue source fields. The LLM trusts these wrong labels and rejects the correct match.

Why Decompiled-C Screening Fails. Ghidra applies an incompatible library definition of `X509`, rendering the serial-number data and length as `signature` and `sig_alg`. The LLM mistakes these inferred names for different digest inputs and rejects the correct function on both reference sides. With no accepted candidate, no paired-function reasoning is invoked: `PatchAlign` treats the target binary as *not affected*, resulting in a MISSING of Table 2 and producing a false negative on the vulnerable target.

Insight. Decompiled C improves overall end-to-end results (Table 2), but incorrect annotations introduce additional noise into LLM analysis. In the assembly-based run, LLM matches the serial-number accesses at offsets `0x8` and `0x10`, accepts the function as *exact*, and subsequent Patch-Status Reasoning follows the reference analysis plan and correctly returns *patched*. Different reference inputs and callee-name handling make this an illustrative comparison. The case motivates treating inferred types as fallible evidence and selectively checking disputed accesses against assembly offsets and data flow.

6 THREATS TO VALIDITY

External Validity. The benchmark contains 62 CVEs from four C/C++ projects, evaluated with GCC `-O0` references and four optimized target settings. Generalization to other projects, build configurations, patch distributions, and LLMs remains unverified. Balancing target labels enables controlled comparison but does not reproduce deployment prevalence or equalize project contributions.

Our evaluation is limited to `x86_64` binaries because the current embedding model supports only this architecture, consistent with the benchmarks used by PS[3] and `React` [40, 41]. Extending `PatchAlign` to other architectures would require compiling vulnerable and patched reference binaries for those architectures and training compatible embedding models; performance in these settings remains unverified. Moreover, LLM reasoning over assembly from less commonly used architectures, such as MIPS, may be less reliable, potentially reducing the accuracy of both Relevance Screening and Patch-Status Reasoning.

Internal Validity. Public repositories, CVEs, and patches may occur in LLM training data [7]. Such prior knowledge is compatible with `PatchAlign`'s 1-day testing setting, which explicitly supplies the source patch; successive model updates can extend this knowledge to newly disclosed vulnerabilities. We also compile and strip target binaries locally under different compiler and

optimization settings, making prior exposure to the exact evaluated assembly less likely. Even with knowledge of the fix, the LLM must determine whether the target implements it.

7 CONCLUSION

`PatchAlign` enables end-to-end patch presence testing in stripped binaries by combining semantic code localization with LLM-driven reasoning about patch-specific behavior. Semantic Localization achieves full instruction coverage within the encoder's limited context size through control-flow windows. It further considers how functions are used by their callers and callees, improving retrieval recall when compiler optimizations reshape function boundaries. Guided by the source patch and both vulnerable and patched references, LLM-driven reasoning verifies the relevance of retrieved code and examines the behavioral distinctions that determine patch status. Together, these capabilities connect robust code retrieval with precise reasoning about small security fixes, without requiring target symbols or a known patch location.

DATA AVAILABILITY

To support reproducibility, we commit to publicly releasing the data, code, models, and instructions to reproduce the evaluation results upon publication.

REFERENCES

[1] Zeina Abu-Aisheh, Romain Raveaux, Jean-Yves Ramel, and Patrick Martineau. 2015. An Exact Graph Edit Distance Algorithm for Solving Pattern Recognition Problems. In *Proceedings of the International Conference on Pattern Recognition Applications and Methods - Volume 1* (Lisbon, Portugal) *(ICPRAM 2015)*. SCITEPRESS - Science and Technology Publications, Lda, Setubal, PRT, 271–278. doi:10.5220/0005209202710278

[2] Omar Abusabha, Jiyong Uhm, Tamer Abuhmed, and Hyungjoon Koo. 2026. A Deep Dive into Function Inlining and its Security Implications for ML-based Binary Analysis. In *Network and Distributed System Security (NDSS) Symposium*. doi:10.14722/ndss.2026.241872

[3] Matt Caswell. 2021. OpenSSL fix for CVE-2021-23841. https://github.com/openssl/openssl/commit/122a19ab48091c657f7cb1fb3af9fc07bd557bbf Commit `122a19a`.

[4] Keith D Cooper, Mary W Hall, and Ken Kennedy. 1993. A Methodology for Procedure Cloning. *Comput. Lang.* 19, 2 (1993), 105–117. doi:10.1016/0096-0551(93)90005-L

[5] Yaniv David and Eran Yahav. 2014. Tracelet-based Code Search in Executables. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '14)*. ACM, 349–360.

[6] DeepSeek. 2026. Models & Pricing. https://api-docs.deepseek.com/quick_start/pricing/

[7] Chunyuan Deng, Yilun Zhao, Xiangru Tang, Mark Gerstein, and Arman Cohan. 2024. Investigating Data Contamination in Modern Benchmarks for Large Language Models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, Mexico City, Mexico, 8706–8719. doi:10.18653/v1/2024.naacl-long.482

[8] Steven HH Ding, Benjamin CM Fung, and Philippe Charland. 2019. Asm2vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler optimization. In *2019 ieee symposium on security and privacy*. IEEE, 472–489.

[9] Ruian Duan, Ashish Bijlani, Yang Ji, Omar Alrawi, Yiyuan Xiong, Moses Ike, Brendan Saltaformaggio, and Wenke Lee. 2019. Automating Patching of Vulnerable Open-Source Software Versions in Application Binaries.. In *NDSS*.

[10] DWARF Debugging Information Format Committee. 2017. *DWARF Debugging Information Format*. https://dwarfstd.org/doc/DWARF5.pdf

[11] Qian Feng, Rundong Zhou, Chengcheng Xu, Yao Cheng, Brian Testa, and Heng Yin. 2016. Scalable graph-based bug search for firmware images. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*. 480–491.

[12] Irfan Ul Haq and Juan Caballero. 2021. A Survey of Binary Code Similarity. *ACM Comput. Surv.* 54, 3, Article 51 (April 2021), 38 pages. doi:10.1145/3446371

[13] Ang Jia, Ming Fan, Wuxia Jin, Xi Xu, Zhaohui Zhou, Qiyi Tang, Sen Nie, Shi Wu, and Ting Liu. 2023. 1-to-1 or 1-to-n? Investigating the Effect of Function Inlining on Binary Similarity Analysis. *ACM Transactions on Software Engineering and Methodology* 32, 4 (2023), 1–26. doi:10.1145/3561385

[14] Ang Jia, Ming Fan, Xi Xu, Wuxia Jin, Haijun Wang, and Ting Liu. 2024. Cross-inlining binary function similarity detection. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13. doi:10.1145/3597503.3639080

[15] Zheyue Jiang, Yuan Zhang, Jun Xu, Qi Wen, Zhenghe Wang, Xiaohan Zhang, Xinyu Xing, Min Yang, and Zhemin Yang. 2020. PDiff: Semantic-based Patch Presence Testing for Downstream Kernels. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security* (Virtual Event, USA) *(CCS '20)*. Association for Computing Machinery, New York, NY, USA, 1149–1163. doi:10.1145/3372297.3417240

[16] Pascal Junod, Julien Rinaldini, Johan Wehrli, and Julie Michielin. 2015. Obfuscator-LLVM: Software Protection for the Masses *(SPRO)*.

[17] Francois-Xavier Le Bail. 2019. tcpdump fix for CVE-2018-14461. https://github.com/the-tcpdump-group/tcpdump/commit/aa5c6b710dfd8020d2c908d6b3bd41f1da719b3b Commit `aa5c6b7`.

[18] Frank Li and Vern Paxson. 2017. A Large-Scale Empirical Study of Security Patches. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security* (Dallas, Texas, USA) *(CCS '17)*. 2201–2215. doi:10.1145/3133956.3134072

[19] Zhijie Liu, Qiyi Tang, Sen Nie, Shi Wu, Liang Feng Zhang, and Yutian Tang. 2025. KEENHash: Hashing programs into function-aware embeddings for large-scale binary code similarity analysis. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 801–824.

[20] Andrea Marcelli, Mariano Graziano, Xabier Ugarte-Pedrero, Yanick Fratantonio, Mohamad Mansouri, and Davide Balzarotti. 2022. How machine learning is solving the binary function similarity problem. In *31st USENIX Security Symposium (USENIX Security 22)*. 2099–2116.

[21] Luca Massarelli, Giuseppe Antonio Di Luna, Fabio Petroni, Roberto Baldoni, and Leonardo Querzoni. 2019. SAFE: Self-attentive function embeddings for binary similarity. In *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, 309–329.

[22] Michel Neuhaus, Kaspar Riesen, and Horst Bunke. 2006. Fast Suboptimal Algorithms for the Computation of Graph Edit Distance. In *Structural, Syntactic, and Statistical Pattern Recognition (Lecture Notes in Computer Science, Vol. 4109)*. Springer, 163–172. doi:10.1007/11815921_17

[23] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748* (2018).

[24] Jannik Pewny, Behrad Garmany, Robert Gawlik, Christian Rossow, and Thorsten Holz. 2015. Cross-Architecture Bug Search in Binary Executables. In *Proceedings of the 2015 IEEE Symposium on Security and Privacy (S&P '15)*. IEEE Computer Society, 709–724.

[25] Rodrigo C. O. Rocha, Pavlos Petoumenos, Zheng Wang, Murray Cole, and Hugh Leather. 2019. Function Merging by Sequence Alignment. In *2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, Washington, DC, USA, 149–163. doi:10.1109/CGO.2019.8661174

[26] Alberto Sanfeliu and King-Sun Fu. 1983. A Distance Measure Between Attributed Relational Graphs for Pattern Recognition. *IEEE Transactions on Systems, Man, and Cybernetics* SMC-13, 3 (1983), 353–362. doi:10.1109/TSMC.1983.6313167

[27] Jingyi Shi, Yufeng Chen, Yang Xiao, Yuekang Li, Zhengzi Xu, Sihao Qiu, Chi Zhang, Keyu Qi, Yeting Li, Xingchu Chen, Yanyan Zou, Yang Liu, and Wei Huo. 2025. A Large Scale Study of AI-based Binary Function Similarity Detection Techniques for Security Researchers and Practitioners. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1070–1082. doi:10.1109/ASE63991.2025.00093

[28] Mark D Smucker, James Allan, and Ben Carterette. 2007. A comparison of statistical significance tests for information retrieval evaluation. In *Proceedings of the sixteenth ACM conference on Conference on information and knowledge management*. 623–632. doi:10.1145/1321440.1321528

[29] Peiyuan Sun, Qiben Yan, Haoyi Zhou, and Jianxin Li. 2021. Osprey: A fast and accurate patch presence test framework for binaries. *Computer Communications* 173 (2021), 95–106. doi:10.1016/j.comcom.2021.03.011

[30] Julián Urbano, Harlley Lima, and Alan Hanjalic. 2019. Statistical significance testing in information retrieval: an empirical analysis of type I, type II and type III errors. In *Proceedings of the 42nd International ACM SIGIR conference on Research and development in information retrieval*. 505–514. doi:10.1145/3331184.3331259

[31] Hao Wang, Zeyu Gao, Chao Zhang, Zihan Sha, Mingyang Sun, Yuchen Zhou, Wenyu Zhu, Wenju Sun, Han Qiu, and Xi Xiao. 2024. CLAP: Learning transferable binary code representations with natural language supervision. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 503–515.

[32] Hao Wang, Zeyu Gao, Chao Zhang, Mingyang Sun, Yuchen Zhou, Han Qiu, and Xi Xiao. 2024. CEBin: A Cost-Effective Framework for Large-Scale Binary Code Similarity Detection. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, Austria) *(ISSTA)*. Association for Computing Machinery, 149–161.

[33] Huaijin Wang and Zhiqiang Lin. 2026. vSim: Semantics-aware value extraction for efficient binary code similarity analysis. In *Network and Distributed Systems Security (NDSS) Symposium*.

[34] Huaijin Wang, Pingchuan Ma, Shuai Wang, Qiyi Tang, Sen Nie, and Shi Wu. 2023. sem2vec: Semantics-aware Assembly Tracelet Embedding. *ACM Transactions on Software Engineering and Methodology* 32, 4, Article 90 (May 2023), 34 pages.

[35] Hao Wang, Wenjie Qu, Gilad Katz, Wenyu Zhu, Zeyu Gao, Han Qiu, Jianwei Zhuge, and Chao Zhang. 2022. jTrans: jump-aware transformer for binary code similarity detection. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. Association for Computing Machinery, 1–13.

[36] Huaijin Wang and Shuai Wang. 2026. Instruction Alignment for Binary Code Representation Learning. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26)*. Munich, Germany. arXiv:2608.11766 [cs.SE] doi:10.1145/3832783.3837516

[37] Xiaojun Xu, Chang Liu, Qian Feng, Heng Yin, Le Song, and Dawn Song. 2017. Neural network-based graph embedding for cross-platform binary code similarity detection. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 363–376.

[38] Yifei Xu, Zhengzi Xu, Bihuan Chen, Fu Song, Yang Liu, and Ting Liu. 2020. Patch based vulnerability matching for binary programs. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis* (Virtual Event, USA) *(ISSTA 2020)*. Association for Computing Machinery, New York, NY, USA, 376–387. doi:10.1145/3395363.3397361

[39] Shouguo Yang, Zhengzi Xu, Yang Xiao, Zhe Lang, Wei Tang, Yang Liu, Zhiqiang Shi, Hong Li, and Limin Sun. 2023. Towards Practical Binary Code Similarity Detection: Vulnerability Verification via Patch Semantic Analysis. *ACM Trans. Softw. Eng. Methodol.* 32, 6, Article 158 (Sept. 2023), 29 pages. doi:10.1145/3604608

[40] Qi Zhan, Xing Hu, Zhiyang Li, Xin Xia, David Lo, and Shanping Li. 2024. PS3: Precise Patch Presence Test based on Semantic Symbolic Signature. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 167, 12 pages. doi:10.1145/3597503.3639134

[41] Qi Zhan, Xing Hu, Xin Xia, and Shanping Li. 2024. REACT: IR-Level Patch Presence Test for Binary. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24)*. Association for Computing Machinery, New York, NY, USA, 381–392. doi:10.1145/3691620.3695012

[42] Hang Zhang and Zhiyun Qian. 2018. Precise and Accurate Patch Presence Test for Binaries. In *27th USENIX Security Symposium (USENIX Security 18)*. USENIX Association, Baltimore, MD, 887–902. https://www.usenix.org/conference/usenixsecurity18/presentation/zhang-hang

[43] Zheng Zhang, Hang Zhang, Zhiyun Qian, and Billy Lau. 2021. An Investigation of the Android Kernel Patch Ecosystem. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 3649–3666. https://www.usenix.org/conference/usenixsecurity21/presentation/zhang-zheng

[44] Fei Zuo, Xiaopeng Li, Patrick Young, Lannan Luo, Qiang Zeng, and Zhexin Zhang. 2019. Neural Machine Translation Inspired Binary Code Similarity Comparison beyond Function Pairs. In *Network and Distributed Systems Security (NDSS) Symposium*.